Wilson — Canonical Entropy Closure BASE

This notebook is the corrected base for the Wilson live packet. It implements the published Tensor P2 entropy-closure simulation directly.

Canonical Tensor P2:

`3,3 / 0,0 / 0,1 / 1,2 / 2,1 / 1,0 / 0,0 / Delay(200)`

Canonical Tensor P3:

`P3 = [P2_1 ⊗ D_1, P2_2 ⊗ D_2, P2_3 ⊗ D_3]`

Published entropy window:

`0,0 → 0,1 → 1,2 → 2,1 → 1,0 → 0,0`

The primary simulation applies depolarizing noise only at `1,0`, never at terminal `0,0`, and evaluates final entropy only after the invariant traversal reaches terminal `0,0`.

The robustness mode applies localized depolarization at every internal node from the first `0,0` through `1,0`, while terminal `0,0` remains unexposed.

This is **not** implemented as one globally noisy density matrix carried unchanged through Tensor P2.

```
!pip -q install numpy matplotlib pandas ipywidgets
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4.9/4.9 MB 58.7 MB/s eta 0:00:00
```

```
import asyncio
import hashlib
import json
from datetime import datetime, timezone

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import ipywidgets as widgets
from IPython.display import display, clear_output


def utc_now():
    return datetime.now(timezone.utc).isoformat()


def sha256_json(obj):
    raw=json.dumps(obj,sort_keys=True,separators=(',',':'),default=str).encode('utf-8')
    return hashlib.sha256(raw).hexdigest()
```

```
from google.colab import userdata

token = userdata.get("IBM_QUANTUM_TOKEN")

print("IBM Quantum token loaded:", bool(token))
```

```
IBM Quantum token loaded: True
```

```
from google.colab import userdata

token = userdata.get("IBM_QUANTUM_TOKEN")
```

1. Fixed recurrence and state roles

The first and terminal `0,0` are the same folded total-state reference reached at different points in the fixed recurrence. They are distinguished below only by their traversal role.

```
P2_FULL = [
    ('3,3', 'bounded triadic entry'),
    ('0,0_ref', 'folded total-state reference after entry'),
    ('0,1', 'differentiated traversal'),
    ('1,2', 'relational transition'),
    ('2,1', 'inversion-paired transition'),
    ('1,0', 'return-directed traversal'),
    ('0,0_close', 'terminal return to folded total-state reference / entropy readout'),
    ('Delay(200)', 'temporal spacing after return'),
]

ENTROPY_WINDOW=['0,0_ref','0,1','1,2','2,1','1,0','0,0_close']

P2_ENCODING={
    '0,0_ref':'000',
    '0,1':'001',
    '1,0':'010',
    '1,2':'011',
    '2,1':'100',
    '3,3':'101',
    '0,0_close':'000',
}

DISPLAY={
    '3,3':'3,3',
    '0,0_ref':'0,0 reference',
    '0,1':'0,1',
    '1,2':'1,2',
    '2,1':'2,1',
    '1,0':'1,0',
    '0,0_close':'0,0 closure/readout',
    'Delay(200)':'Delay(200)',
}

P3_CANONICAL='P3 = [P2_1 ⊗ D_1, P2_2 ⊗ D_2, P2_3 ⊗ D_3]'
```

2. Source-faithful entropy-closure engine

Three-qubit Hilbert-space encoding uses an 8-dimensional density matrix. The primary depolarizing exposure is:

`ρ → (1-p)ρ + pI/8`

at `1,0` only.

Each P2 label is evaluated at its canonical encoded density state. Noise is localized to the designated traversal node(s). Terminal `0,0` is reached by the fixed traversal and receives no noise; the final readout is taken there.

```
D=8
I8=np.eye(D,dtype=complex)


def basis_density(bits):
    idx=int(bits,2)
    ket=np.zeros((D,1),dtype=complex)
    ket[idx,0]=1.0
    return ket@ket.conj().T


def depolarize(rho,p):
    p=float(np.clip(p,0.0,1.0))
    return (1-p)*rho + p*I8/D


def metrics(rho):
    rho=np.asarray(rho,dtype=complex)
    trace=float(np.real_if_close(np.trace(rho)))
    purity=float(np.real_if_close(np.trace(rho@rho)))
    eig=np.linalg.eigvalsh((rho+rho.conj().T)/2).real
    eig[np.abs(eig)<1e-14]=0.0
    eig=np.clip(eig,0.0,None)
    entropy=float(-sum(x*np.log2(x) for x in eig if x>0))
    return {'trace':trace,'purity':purity,'entropy_bits':entropy,'eigenvalues':eig.tolist()}


def exposure_set(mode):
    if mode=='primary_1_0':
        return {'1,0'}
    if mode=='all_internal':
        return {'0,0_ref','0,1','1,2','2,1','1,0'}
    raise ValueError(mode)


def entropy_closure_cycle(p=0.01,mode='primary_1_0'):
    exposed_states=exposure_set(mode)
    rows=[]
    for order,state in enumerate(ENTROPY_WINDOW):
        rho=basis_density(P2_ENCODING[state])
        exposed=(state in exposed_states and state!='0,0_close')
        rho_local=depolarize(rho,p) if exposed else rho
```

```
            m=metrics(rho_local)
            rows.append({
                'order':order,
                'state':state,
                'display_state':DISPLAY[state],
                'encoding':P2_ENCODING[state],
                'noise_exposed':exposed,
                'p':float(p) if exposed else 0.0,
                'entropy_bits':m['entropy_bits'],
                'purity':m['purity'],
                'trace':m['trace'],
                'eigenvalues':m['eigenvalues'],
            })
        final=rows[-1]
        result={
            'timestamp_utc':utc_now(),
            'mode':mode,
            'p':float(p),
            'exposed_states':sorted(exposed_states),
            'records':rows,
            'final_state':final['state'],
            'final_trace':final['trace'],
            'final_purity':final['purity'],
            'final_entropy_bits':final['entropy_bits'],
            'final_eigenvalues':final['eigenvalues'],
        }
        result['sha256']=sha256_json(result)
        return result
```

3. Hard validation against the supplemental record

The notebook will stop here if the implementation no longer reproduces the source design.

```
primary=entropy_closure_cycle(0.01,'primary_1_0')
assert primary['exposed_states']==['1,0']
assert primary['final_state']=='0,0_close'
assert abs(primary['final_trace']-1.0)<1e-12
assert abs(primary['final_purity']-1.0)<1e-12
assert abs(primary['final_entropy_bits'])<1e-12

robust=entropy_closure_cycle(0.10,'all_internal')
max_intermediate=max(r['entropy_bits'] for r in robust['records'][:-1])
assert abs(max_intermediate-0.6737131849093467)<1e-9
assert abs(robust['final_entropy_bits'])<1e-12

print('SOURCE ALIGNMENT: PASS')
print('Primary p=0.01 entropy at 1,0:', next(r['entropy_bits'] for r in primary['records'] if r['state']=='1,0'))
print('All-internal p=0.10 max intermediate entropy:', max_intermediate)
print('Terminal 0,0 entropy:', robust['final_entropy_bits'])
print('Terminal eigenvalue spectrum:', robust['final_eigenvalues'])
```

```
SOURCE ALIGNMENT: PASS
Primary p=0.01 entropy at 1,0: 0.09695193330815613
All-internal p=0.10 max intermediate entropy: 0.6737131849093467
Terminal 0,0 entropy: -0.0
Terminal eigenvalue spectrum: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0]
```

4. Continuous Tensor P2 entropy-closure loop

This display continuously traverses the full P2 order. The entropy window itself remains the six-state published window. `3,3` precedes it and `Delay(200)` follows terminal return.

```
STATE={'running':False,'stop':False,'cycle':0,'idx':0,'current':None}
HISTORY=[]
TASK={'task':None}

mode_w=widgets.Dropdown(options=[('Primary: noise only at 1,0','primary_1_0'),('Robustness: noise at all internal nodes','all_internal')],value='primary_1_0',description='Mode',layout=widgets.Layout(width='95%'))
p_w=widgets.FloatSlider(value=0.01,min=0,max=0.10,step=0.001,description='p',readout_format='.3f',continuous_update=False,layout=widgets.Layout(width='95%'))
ms_w=widgets.IntSlider(value=350,min=80,max=1600,step=20,description='Step ms',continuous_update=False,layout=widgets.Layout(width='95%'))
toggle=widgets.ToggleButton(value=False,description='Start loop',icon='play',button_style='primary')
once=widgets.Button(description='Run one cycle',icon='step-forward')
clear=widgets.Button(description='Clear',icon='trash')
status=widgets.HTML(); plot_out=widgets.Output(); table_out=widgets.Output()


def current_row():
    if not STATE['current']: return None
    state=P2_FULL[STATE['idx']][0]
    if state not in ENTROPY_WINDOW: return None
    return next((r for r in STATE['current']['records'] if r['state']==state),None)


def render():
    state,role=P2_FULL[STATE['idx']]
    row=current_row()
    s=row['entropy_bits'] if row else 0.0
    exp=row['noise_exposed'] if row else False
    final=STATE['current']['final_entropy_bits'] if STATE['current'] else None
    status.value=(
        '<div style="padding:12px;border:1px solid #bbb;border-radius:10px">'
        f'<b>Tensor P2 Entropy Closure</b> · cycle <b>{STATE["cycle"]}</b><br>'
        f'P2 state <b>{DISPLAY[state]}</b> · {role}<br>'
        f'Localized exposure <b>{"YES" if exp else "NO"}</b> · local S <b>{s:.9f} bits</b><br>'
        f'Terminal 0,0 S <b>{"—" if final is None else f"{final:.9f} bits"}</b>'
        '</div>'
    )

    with plot_out:
        clear_output(wait=True)
        fig=plt.figure(figsize=(14,7))
        gs=fig.add_gridspec(2,2,height_ratios=[0.8,1.2])
        ax=fig.add_subplot(gs[0,:])
        x=np.arange(len(P2_FULL)); y=np.zeros(len(x))
        ax.plot(x,y,linewidth=1.5); ax.scatter(x,y,s=80); ax.scatter([STATE['idx']],[0],s=260,zorder=5)
        for j,(st,_) in enumerate(P2_FULL): ax.text(j,.045 if j%2==0 else -.045,DISPLAY[st],ha='center',va='center',rotation=25,fontsize=8)
        ax.set_xlim(-.4,len(P2_FULL)-.6); ax.set_ylim(-.16,.16); ax.set_xticks([]); ax.set_yticks([]); ax.set_title('Invariant Tensor P2 recurrence')

        ax=fig.add_subplot(gs[1,0])
        # Structural P3 nesting only; no invented weights.
        theta=np.linspace(0,2*np.pi,360); radii=[1.0,1.45,1.9]; labels=['P2₁ ⊗ D₁','P2₂ ⊗ D₂','P2₃ ⊗ D₃']
        ang=2*np.pi*STATE['idx']/(len(P2_FULL)-1)
        for r,label in zip(radii,labels):
            ax.plot(r*np.cos(theta),r*np.sin(theta)); ax.scatter([r*np.cos(ang)],[r*np.sin(ang)],s=100); ax.text(r+.05,0,label,fontsize=9,va='center')
        ax.axvline(0,linestyle='--',linewidth=1); ax.text(.05,1.98,'M_E',fontsize=9); ax.set_aspect('equal'); ax.set_axis_off(); ax.set_title(P3_CANONICAL+'\nstructural embedding')

        ax=fig.add_subplot(gs[1,1])
        if STATE['current']:
            rows=STATE['current']['records']; vals=[r['entropy_bits'] for r in rows]; labs=[r['display_state'] for r in rows]
            ax.plot(range(len(vals)),vals,marker='o')
            for j,r in enumerate(rows):
                if r['noise_exposed']: ax.scatter([j],[vals[j]],s=150,marker='s')
            ax.set_xticks(range(len(labs)),labs,rotation=30); ax.set_ylabel('Von Neumann entropy (bits)'); ax.set_title('Published entropy-closure window'); ax.grid(alpha=.25)
        else:
            ax.text(.5,.5,'Run a cycle',ha='center',va='center',transform=ax.transAxes); ax.set_axis_off()
        plt.tight_layout(); plt.show()

    with table_out:
        clear_output(wait=True)
        if STATE['current']:
            display(pd.DataFrame([{'state':r['display_state'],'encoding':r['encoding'],'noise':r['noise_exposed'],'p':r['p'],'entropy_bits':r['entropy_bits'],'purity':r['purity'],'trace':r['trace']} for r in STATE['current']['records']]))


async def run_cycle(animated=True):
    STATE['cycle']+=1
    STATE['current']=entropy_closure_cycle(p_w.value,mode_w.value)
    for idx in range(len(P2_FULL)):
        if STATE['stop']: break
        STATE['idx']=idx; render()
        if animated: await asyncio.sleep(ms_w.value/1000)
    if not STATE['stop']:
        HISTORY.append(STATE['current'])
        if len(HISTORY)>300: del HISTORY[:-300]
        render()


async def loop_worker():
    try:
        while toggle.value and not STATE['stop']:
            await run_cycle(True)
    except asyncio.CancelledError:
        pass
    except Exception as exc:
        status.value=f'<b>Loop error:</b> {exc}'
    finally:
        STATE['running']=False
        if toggle.value: toggle.value=False
        toggle.description='Start loop'; toggle.icon='play'


def on_toggle(change):
```

```
    if change['name']!='value': return
    if change['new']:
        STATE['stop']=False; STATE['running']=True; toggle.description='Pause loop'; toggle.icon='pause'
        TASK['task']=asyncio.get_running_loop().create_task(loop_worker())
    else:
        STATE['stop']=True; toggle.description='Start loop'; toggle.icon='play'


def on_once(_):
    if not STATE['running']:
        STATE['stop']=False; asyncio.get_running_loop().create_task(run_cycle(True))


def on_clear(_):
    if STATE['running']: return
    HISTORY.clear(); STATE.update({'stop':False,'cycle':0,'idx':0,'current':None}); render()


toggle.observe(on_toggle,names='value'); once.on_click(on_once); clear.on_click(on_clear)
display(widgets.VBox([mode_w,p_w,ms_w,widgets.HBox([toggle,once,clear]),status,plot_out,table_out]))
render()
```

Mode: Primary: noise only at 1,0

p: 0.096

Step ms: 350

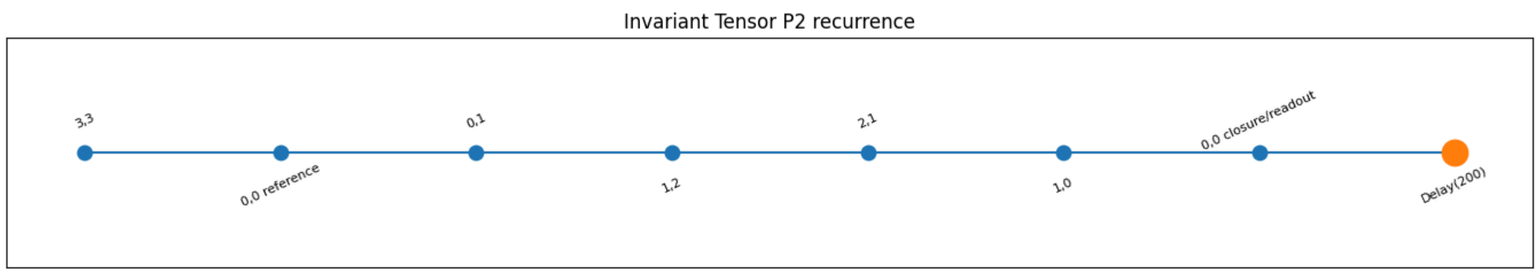
Pause loop | Run one cycle | Clear

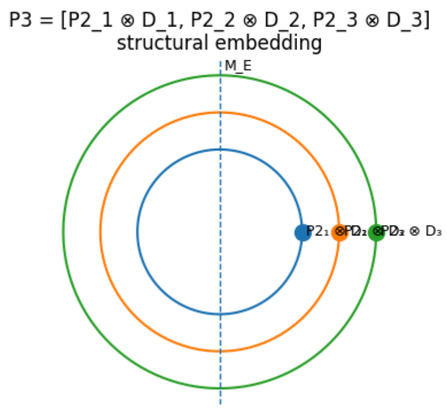
Tensor P2 Entropy Closure · cycle **2**

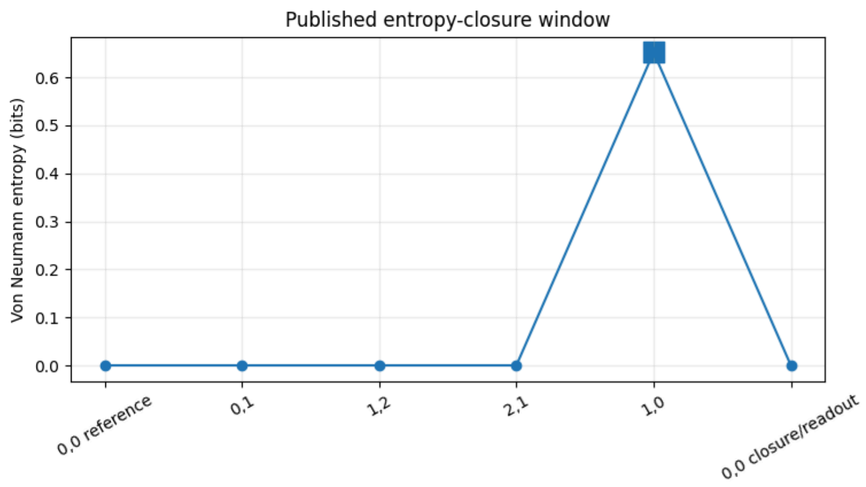
P2 state **Delay(200)** · temporal spacing after return

Localized exposure **NO** · local S **0.000000000 bits**

Terminal 0,0 S **-0.000000000 bits**







	state	encoding	noise	p	entropy_bits	purity	trace
0	0,0 reference	000	False	0.000	-0.000000	1.000000	1.0
1	0,1	001	False	0.000	-0.000000	1.000000	1.0
2	1,2	011	False	0.000	-0.000000	1.000000	1.0
3	2,1	100	False	0.000	-0.000000	1.000000	1.0
4	1,0	010	True	0.096	0.651937	0.840064	1.0
5	0,0 closure/readout	000	False	0.000	-0.000000	1.000000	1.0

5. Scope of this corrected BASE

This file deliberately stops at the **canonical entropy-closure engine + continuous P2/P3 structural visualization**.

It does not yet re-add the prior AI/QRTM/ethics orchestration, because those layers should be attached only after the entropy-closure base is correct. In particular, IBM/QPU measurement counts are not treated as the entropy payload of this density-state simulation.